

AMO 结构保持算子：中文版算法框架与伪代码

（依据 PlatEMO A-amos 实现整理）

符号约定：染色体长度 $varDim = 2n + m - 1$ ；基因 $1..n$ 为商家， $n+1..2n$ 为客户；共 $m-1$ 个分隔符 “0”；每个商家 s 必须出现在客户 $s+n$ 之前。
种群初始化不在 `OperatorAMO` 内完成。
PlatEMO 中由 `Problem.Initialization()` 生成初代（参见 `A_amos.m`）。
本文档侧重遗传算子模块。

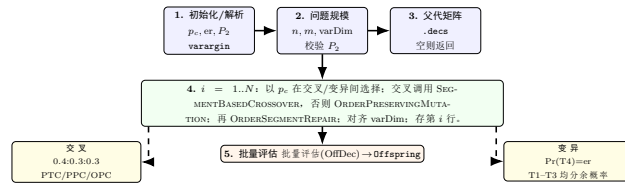


图 1: OperatorAMO 整体框架（与 OperatorAMO.m 一致）：参数解析、问题字段、逐个体遗传操作、统一修复与批量评估；侧框概括三种交叉与四种变异。

Algorithm 1: OperatorAMO (1/2) 初始化与输入, 对应 OperatorAMO.m 前半段

输入: Problem, Parents.decs, varargin

输出: 规模 n, m, varDim , 矩阵 ParentDec , P_2^{dec} , 超参 p_c, er ; 或空则直接返回

1 $p_c \leftarrow 0.5$; $\text{er} \leftarrow 0.25$; $P_2 \leftarrow \emptyset$; 按键值对解析 varargin 更新

p_c, er, P_2 (键名 pc / er / p2 等)

2 $n, m, \text{varDim} \leftarrow \text{Problem.n}, \text{Problem.m}, \text{Problem.D}$;

$\text{ParentDec} \leftarrow \text{Parents.decs}$

3 if ParentDec 为空 then

4 return 批量评估(ParentDec)

5 $P_2^{\text{dec}} \leftarrow$ 若 P_2 非空则取其决策矩阵并校验与 ParentDec 同形, 否则 []

Algorithm 2: OperatorAMO (2/2) 逐个体生成子代并批量评估

输入: $ParentDec, P_2^{dec}, n, m, \text{varDim}, p_c, \text{er}$
 输出: Offspring

```

1  $N \leftarrow \text{nrows}(ParentDec); \text{OffDec} \leftarrow \mathbf{0}^{N \times \text{varDim}}$ 
2 for  $i = 1$  to  $N$  do
3    $p_1 \leftarrow ParentDec(i, :)$ 
4   if  $\text{rand}() < p_c$  且  $N \geq 2$  then
5      $p_2 \leftarrow$  若  $P_2^{dec} \neq []$  则  $P_2^{dec}(i, :)$ , 否则在  $ParentDec$  中随机另
      取一行
6      $c \leftarrow \text{SEGMENTBASEDCROSSOVER}(p_1, p_2, n, m, \text{varDim})$ 
7   else
8      $c \leftarrow \text{ORDERPRESERVINGMUTATION}(p_1, m, \text{varDim}, n, \text{er})$ 
9    $c \leftarrow \text{ORDERSEGMENTREPAIR}(c, n, m, \text{varDim})$ ; 对齐长度
       $\text{varDim}; \text{OffDec}(i, :) \leftarrow c$ 
10 return 批量评估( $\text{OffDec}$ )
  
```

Algorithm 3: 分段交叉入口 SegmentBasedCrossover

输入: 父代 p_1, p_2 , 规模 n, m, varDim
 输出: 子代 c

```

1  $r \leftarrow \text{rand}()$ 
2 if  $r < 0.4$  then
3    $c \leftarrow \text{POSITIONTRACKINGCROSSOVER}(\dots)$  // PTC 位置追踪
4 else if  $r < 0.7$  then
5    $c \leftarrow \text{PAIRPRIORITYCROSSOVER}(\dots)$  // PPC 对优先级
6 else
7    $c \leftarrow \text{ORDERPRESERVINGCROSSOVER}(\dots)$  // OPC 顺序保持
8 将  $c$  对齐至  $\text{varDim}$ ; 去除首尾 0
9 if  $c = p_1$  或  $c = p_2$  then
10    $c \leftarrow \text{FORCEDISRUPT}(c, n, m, \text{varDim})$  // 打破克隆
11 return  $c$ 
  
```

Algorithm 4: 位置追踪交叉 PTC
(PositionTrackingCrossover)

输入: 合法父代 p_1, p_2

输出: 子代 c

```

1 建立基因  $1..2n$  的位置映射  $\pi_1, \pi_2$ 
2  $\mathcal{K} \leftarrow \{k : k \text{ 对 } \pi_1, \pi_2 \text{ 均为安全切点} \}$ 
   // 安全: 不存在  $(s, s+n)$  使  $\pi(s) < k < \pi(s+n)$ 
3 if  $\mathcal{K} = \emptyset$  then
4   return  $p_1$ 
5 均匀抽取  $k \in \mathcal{K}$ 
6  $c(1:k) \leftarrow p_1(1:k)$ ;  $U \leftarrow c(1:k)$  中已出现基因集合
7 for 自左向右扫描  $p_2$  的下标  $t$  do
8    $g \leftarrow p_2(t)$ 
9   if  $g \notin U$  then
10    if  $g = 0$  then
11      追加  $g$ ; 加入  $U$ 
12    else if  $n < g \leq 2n$  then
13      if 商家  $(g-n)$  已在  $U$  中 then
14        追加  $g$ ; 加入  $U$ 
15    else
16      追加  $g$ ; 加入  $U$ 
17  if 已填满长度  $varDim$  then
18    跳出
19  $c \leftarrow \text{REPAIRSOLUTION}(c, n, m, varDim)$ 
20 if 不满足  $ISVALID(c)$  then
21    $c \leftarrow \text{REPAIRORDER}(c, n)$ 
22  $c \leftarrow \text{REPAIRSEGMENTS}(c, n, m)$ 
23 return  $c$ 

```

Algorithm 5: 对优先级交叉 PPC (PairPriorityCrossover)

输入: 合法父代 p_1, p_2

输出: 子代 c

```

1 for  $s = 1$  to  $n$  do
2    $\text{pri}_1(s) \leftarrow s$  在  $p_1$  中首次出现位置
3    $\text{pri}_2(s) \leftarrow s$  在  $p_2$  中首次出现位置
4    $\text{merged}(s) \leftarrow \min(\text{pri}_1(s), \text{pri}_2(s))$ 
5 按  $\text{merged}$  升序排列商家得列表  $M$ 
6  $c \leftarrow \text{zeros}(1, \text{varDim})$ ; 从左起依次写入  $M$  中商家
7 for 列表  $M$  中每个商家  $s$  do
8   在  $c$  中找到  $s$  的位置  $\ell$ 
9   自  $\ell+1$  向右扫描: 将第一个仍为 0 的位置改为客户  $s+n$ 
10  $c \leftarrow \text{REPAIRSOLUTION}(c, n, m, \text{varDim})$ 
11 if 不满足  $\text{ISVALID}(c)$  then
12    $c \leftarrow \text{REPAIRORDER}(c, n)$ 
13  $c \leftarrow \text{REPAIRSEGMENTS}(c, n, m)$ 
14 return  $c$ 

```

Algorithm 6: 顺序保持交叉 OPC
(OrderPreservingCrossover)

输入: 合法父代 p_1, p_2

输出: 子代 c

- 1 提取商家访问顺序 O_1, O_2 (按扫描首次出现, 去重保序)
 - 2 **if** O_1 或 O_2 为空 **then**
 - 3 **return** p_1
 - 4 $f \leftarrow \text{randint}(1, |O_1|)$
 - 5 $F \leftarrow \text{unique}([O_1(1:f), O_2(f+1:\text{end})], \text{stable})$
 - 6 将 $O_1 \cup O_2$ 中尚未出现在 F 的商家接到 F 末尾
 - 7 **if** $F = O_1$ 且 $|F| \geq 2$ **then**
 - 8 随机交换 F 中两个位置
 - 9 按与 PPC 相同方式由 F 构造 c (先写商家, 再在各自后填第一个 0 为客户)
 - 10 $c \leftarrow \text{REPAIRSOLUTION}(c, n, m, \text{varDim})$
 - 11 **if** 不满足 $\text{IsValid}(c)$ **then**
 - 12 $c \leftarrow \text{REPAIRORDER}(c, n)$
 - 13 $c \leftarrow \text{REPAIRSEGMENTS}(c, n, m)$
 - 14 **return** c
-

Algorithm 7: 保序变异 OrderPreservingMutation (调度 T1–T4)

输入: 个体 x , m, n, varDim , 探索率 ρ (即 **er**)
输出: 变异后 x'

- 1 用 $m-1$ 个 0 将 x 切分为段 $S_1..S_m$
- 2 **if** $\text{rand}() < \rho$ **then**
- 3 $x' \leftarrow \text{TYPE4_SEGSWAP}(x, \{S_k\}, m, n, \text{varDim})$ // T4 大步长:
 跨段迁移整对
- 4 **else**
- 5 $t \leftarrow \text{randint}(1, 3)$
- 6 **if** $t = 1$ **then**
- 7 $x' \leftarrow \text{TYPE1}(x)$ // T1: 段内商家移至段首
- 8 **else if** $t = 2$ **then**
- 9 $x' \leftarrow \text{TYPE2}(x)$ // T2: 段内客户移至段尾
- 10 **else**
- 11 $x' \leftarrow \text{TYPE3_MOVEDELIMITER}(x)$ // T3: 滑动分隔符 0
- 12 修正长度与 0 的个数; **if** $x' = x$ **then**
- 13 再试 T4 或段内随机对交换
- 14 **return** x'

Algorithm 8: 变异 T1: 随机商家移至该段段首

输入: x , 段 $\{S_k\}$, 扩展零位置数组 E
输出: x'

- 1 $\mathcal{V} \leftarrow$ 含至少一个商家基因 $\{1..n\}$ 的段号集合
- 2 **if** $\mathcal{V} = \emptyset$ **then**
- 3 **return** x
- 4 随机取 $k \in \mathcal{V}$, $\sigma \leftarrow S_k$
- 5 随机取 σ 中一商家基因 a
- 6 从 σ 原位移除 a ; $\sigma' \leftarrow [a, \sigma]$
- 7 $x' \leftarrow x$; 将 σ' 写回段 k 对应全局位置
- 8 **return** x'

Algorithm 9: 变异 T2: 随机客户移至该段段尾

输入: x , 段 $\{S_k\}$, 扩展索引 E

输出: x'

- 1 $\mathcal{V} \leftarrow$ 含至少一个客户基因 $\{n+1..2n\}$ 的段号集合
 - 2 **if** $\mathcal{V} = \emptyset$ **then**
 - 3 **return** x
 - 4 随机取 $k \in \mathcal{V}$, $\sigma \leftarrow S_k$
 - 5 随机取客户 b ; 移除 b ; $\sigma' \leftarrow [\sigma, b]$
 - 6 $x' \leftarrow x$; 写回段 k
 - 7 **return** x'
-

Algorithm 10: 变异 T3: 在不撕开完整对的前提下滑动一个分隔符 0

输入: x , $\{S_k\}$, E , m, n, varDim

输出: x'

- 1 **if** $m \leq 1$ **then**
 - 2 **return** x
 - 3 随机选可移动的段间分隔符编号 $z \in \{1..m-1\}$
 - 4 $d \leftarrow \text{randint}(0, 1)$ // 0: 向左段扫描; 1: 向右段扫描
 - 5 **if** $d = 0$ **then**
 - 6 $q \leftarrow \text{TRAVERSEFORWARD}(\dots)$ // 商家前合法插 0
 - 7 **else**
 - 8 $q \leftarrow \text{TRAVERSEBACKWARD}(\dots)$ // 客户后合法插 0
 - 9 **if** $q < 0$ **then**
 - 10 **return** x
 - 11 删去原位置 $E(z+1)$ 处的 0; 必要时修正 q
 - 12 在全局位置 q 插入 0; 对齐 varDim
 - 13 **return** x'
-

Algorithm 11: T3 辅助: TraverseForward / TraverseBackward

(要点)

输入: 段向量、分隔符上下文、 n
输出: 新 0 的全局下标, 失败返回 -1

- 1 TRAVERSEFORWARD: 自左段尾向头扫描
- 2 在每个商家 a 处, 设 C 为已扫后缀中的客户集合
- 3 **if** C 为空或 C 只属于已扫商家与 $\{a\}$ 对应客户 且 a 前位置合法 **then** 返回该下标
- 4 TRAVERSEBACKWARD: 自右段头向尾扫描
- 5 在每个客户 c 处, 设 M 为已扫前缀中的商家集合
- 6 **if** M 为空或 M 只属于已扫客户与 $\{c\}$ 对应商家 且 c 后位置合法 **then** 返回该下标
- 7 **return** -1

Algorithm 12: 变异 T4: Type4_SegSwap (跨段迁移完整商家-客户对, 示意)

输入: 段划分 $\{S_k\}$

- 1 选取至少含两对的源段 k_s
- 2 随机选源段中成对出现的 $(a, a+n)$
- 3 从 S_{k_s} 按序移除 $(a, a+n)$
- 4 随机选目标段 $k_t \neq k_s$; 在 S_{k_t} 随机插入位置连续插入 $a, a+n$
- 5 重建染色体; **if** 约束不满足 **then**
- 6 **return** 未改变的 x
- 7 **return** 重建个体

Algorithm 13: 段修复流水线 OrderSegmentRepair (抽象)

输入: 染色体 c

- 1 ENSURENOZEROATENDS(c) // 首尾非 0
- 2 REPAIRSEGMENTS(c) // 客户归位到对应商家所在段
- 3 REPAIRSEGMENTSIZES(c, m) // 每段长度偶数且 ≥ 2
- 4 长度对齐至 varDim
- 5 **return** c
